

Glass Workflow Automation - Development Report & Interview Q&A

The build process, the questions it invites, terms answered, and sample code

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the 109-commit development history

Contents

How to use this report	3
Part 1 - The development story	4
Phase 1 - Backend foundation and auth (mid-Jun 2026)	4
Phase 2 - Core modules	4
Phase 3 - The frontend platform	4
Phase 4 - The diagram editor (the centrepiece)	4
Phase 5 - Workflow and production	4
Phase 6 - Quotation, reporting, retention	5
Part 2 - Questions and detailed answers	6
The diagram feature - the honest answer (read this carefully)	6
“Why did you migrate from Prisma to direct PostgreSQL?”	6
“How do you cut a hole in a shape - what is a boolean operation?”	6
“What is bezier curve-fitting, and why?”	6
“How does the geometry validation work?”	7
“What is undo/redo, and how did you implement it?”	7
“What is TanStack Query?”	7
“What is the S3 failover in your file upload?”	7
“What is a layered architecture?”	7
Part 3 - The bait-terms dictionary	8
Part 4 - Sample code snippets (pen and paper)	10
The hooks to drop on purpose	12

How to use this report

Glass Workflow Automation (built for Modern Mahal) digitises a glass-manufacturing business - from order to production tracking - and its centrepiece is a genuinely sophisticated **interactive diagram editor** with geometry validation. It is your proof of applying real engineering to a messy business problem. Structure: **Part 1** build story, **Part 2** Q&A, **Part 3** bait-terms dictionary, **Part 4** sample code.

What it is in one breath: a platform where a glass business turns customer orders into precise, editable production diagrams, validates the geometry so a bad measurement never reaches the factory, generates quotations, and tracks each job through production.

Read the diagram honesty note in Part 2 before the interview - there's a resume claim to get straight.

Part 1 - The development story

Phase 1 - Backend foundation and auth (mid-Jun 2026)

- **Node.js** + **Express** backend using ES modules, with **ESLint** + **Prettier** and a **layered architecture**.
- Started with the **Prisma** ORM, then **migrated to direct PostgreSQL** using **node-pg-migrate** for schema migrations.
- **Argon2** password hashing; a **BigInt serialization** utility; centralised error handling with an **asyncHandler**.
- **JWT** auth with cookies and protected routes; logout with **token blacklisting**; an auth **middleware** and a **role middleware** for RBAC.
- **Google Authenticator** two-factor auth (TOTP/OTP); session management and token cleanup.

Phase 2 - Core modules

- User-management and staff-management modules; role-specific sign-in.
- **Database optimisation** - indexes and sorting.
- Testing infrastructure with **Vitest**.
- A file-upload service with **S3 failover** and a local-filesystem fallback.

Phase 3 - The frontend platform

- **React** frontend with a centralised API client wired to **TanStack Query**.
- Reusable **table**, **modal**, **pagination**, and **filter** components; a sidebar/nav dashboard with analytics cards.
- User, staff, and activity-monitoring modules.

Phase 4 - The diagram editor (the centrepiece)

- An **interactive canvas** editor: shapes, **hinges/handles**, vertices, a **pen** tool, and **cut** tools, with **validation markers**.
- **Boolean geometry** for holes and cuts using a **polygon-clipping** library (union/difference).
- **Bezier curve-fitting**: boolean/cut results are simplified into smooth curved paths; per-ring vertex and curve editing.
- **Editable dimensions** on every ring; cut-tool snapping; closed-lasso cuts; two cut modes; subpath-safe moves.
- **Multi-page** diagrams with **undo/redo**, **autosave** drafts, and named **checkpoints**; page-aware **SVG** and **PDF** export.
- **Validation** for hinges, handles, ellipses, and paths - the geometry/dimension/feasibility checks.

Phase 5 - Workflow and production

- **Workflow templates** and **stage management**; a production **kanban**, manufacturing detail, a template builder, and a staff view.
- **Production aggregates** on the dashboard.

Phase 6 - Quotation, reporting, retention

- A **quotation** backend: pricing, lifecycle, and **PDF/Excel export** (logo embedded in the quotation PDF).
 - **Reporting** endpoints with date ranges and Excel export.
 - **30-day retention triggers** to auto-clean activity logs and notifications.
-

Part 2 - Questions and detailed answers

The diagram feature - the honest answer (read this carefully)

There's a resume claim to get straight before the interview. Your resume says you “built AI-assisted diagram processing that converts rough customer sketches received over WhatsApp into structured, fully editable production diagrams.”

The commit history strongly supports “**structured, fully editable production diagrams**” and “**smart validation for geometry, dimension, and manufacturing feasibility**” - that interactive editor is real and impressive. What the commits I can see do **not** clearly show is the “**AI-assisted**” conversion of “WhatsApp” sketches part.

- If the AI/WhatsApp intake genuinely exists (in another repo, or via an external service), be ready to describe how it works - what model or method reads a sketch, and how it hands off to the editor.
- If it does **not** exist yet, do **not** claim it. Lead instead with what is genuinely yours and genuinely strong: “I built an interactive production-diagram editor with real geometry validation - boolean cuts, curved holes, dimension checks - so a wrong measurement is caught before it reaches the factory.” That is a *better* answer than a vague AI claim you can't defend.
- Consider softening the resume line to match reality (e.g. “structured, fully editable production diagrams with smart geometry validation”) so there's nothing to get caught on.

The editor below is your real, defensible strength - lead with it.

“Why did you migrate from Prisma to direct PostgreSQL?”

Prisma is a great ORM, but for this project I wanted more direct control over the SQL and the migrations, and to avoid the ORM's abstraction where the queries got complex. I moved to writing SQL directly with node-pg-migrate handling versioned schema changes. It is the same “right tool for the job” judgement - the ORM's convenience was costing me control I needed here.

“How do you cut a hole in a shape - what is a boolean operation?”

A boolean operation combines two shapes: union merges them, difference subtracts one from the other. To cut a hole in a glass panel, I take the difference of the panel polygon and the hole polygon using a polygon-clipping library. The result can be a multi-ring path - the outer boundary plus the hole boundaries - which I then validate and export.

“What is bezier curve-fitting, and why?”

Boolean operations output lots of tiny straight segments. Bezier curve-fitting simplifies those into a few smooth curved paths, so a rounded cut looks and exports as a clean curve rather than a jagged polyline. It makes the diagrams accurate and production-ready.

“How does the geometry validation work?”

On save, it checks things a real factory cares about: dimensions that do not add up, holes that overlap or sit outside the panel, hinges or handles placed impossibly, and shapes that are not manufacturable. It flags them with markers on the canvas so the operator sees the problem while they still have context - before the job reaches the floor, where a mistake is expensive.

“What is undo/redo, and how did you implement it?”

The editor keeps a history of states. Undo moves the pointer back to a previous state; redo moves it forward. Combined with autosave drafts and named checkpoints, an operator can experiment freely and never lose work.

“What is TanStack Query?”

A data-fetching and caching library for React. It handles fetching, caching, background refetching, and loading/error states, so the dashboard’s data stays fresh without me hand-managing it. It removes a lot of boilerplate around server state.

“What is the S3 failover in your file upload?”

Uploads go to S3 (cloud object storage), but if S3 is unavailable the service **fails over** to the local filesystem so an upload never just fails. It is a resilience pattern - degrade gracefully rather than break.

“What is a layered architecture?”

Separating the code into layers - routes handle HTTP, controllers coordinate, services hold business logic, and a data layer talks to the database - so each layer has one job and they are easy to test and change independently.

Part 3 - The bait-terms dictionary

Express / ES modules / layered architecture

A minimal Node web framework; modern module syntax; separated route/service/data layers. *Say it:* “Express with a layered architecture - routes, services, data - each with one job.”

Prisma -> node-pg-migrate

Moving from an ORM to direct SQL with versioned migrations. *Say it:* “I moved off Prisma to direct Postgres for control over complex queries.”

Argon2 / JWT / token blacklist / TOTP 2FA / RBAC

Password hashing, signed tokens, revocation list, time-based 2FA, role checks. *Say it:* “Argon2 hashing, JWT with a blacklist for logout, TOTP 2FA, and role middleware.”

asyncHandler

A wrapper that forwards async errors to one error handler. *Say it:* “An asyncHandler wraps routes so async errors hit one central handler.”

Boolean geometry / polygon-clipping

Union/difference of shapes to cut holes. *Say it:* “Holes are a polygon difference via a clipping library, producing multi-ring paths.”

Bezier curve-fitting

Simplifying many segments into smooth curves. *Say it:* “I curve-fit boolean results into smooth beziers so cuts export clean.”

Validation markers

On-canvas flags for geometry/dimension problems. *Say it:* “Validation flags bad dimensions and overlapping holes before production.”

Undo/redo (history stack)

State history with a movable pointer. *Say it:* “Undo/redo is a state-history stack, plus autosave and checkpoints.”

SVG / PDF export

Vector and print export of diagrams. *Say it:* “Diagrams export to SVG and PDF, page-aware.”

TanStack Query

React server-state fetching/caching. *Say it:* “TanStack Query manages fetching and caching on the dashboard.”

S3 failover

Cloud storage with local fallback. *Say it:* “Uploads go to S3, failing over to local disk if S3 is down.”

Indexing / retention triggers

Query speedups; scheduled auto-cleanup. *Say it:* “Indexes for speed, and 30-day retention triggers to purge old logs.”

Kanban / workflow templates

Board-based production tracking; reusable stage flows. *Say it:* “Production is a kanban driven by workflow templates.”

Part 4 - Sample code snippets (pen and paper)

Cutting a hole - a boolean difference (conceptual)

```
import polygonClipping from "polygon-clipping";

function cutHole(panel, hole) {
  // difference = panel minus hole -> outer ring + hole ring(s)
  return polygonClipping.difference(panel, hole);
}
```

Say aloud: "A hole is the panel polygon minus the hole polygon - a boolean difference - giving a multi-ring path."

Geometry validation

```
function validate(panel, holes) {
  const errors = [];
  for (const h of holes) {
    if (!insideBounds(h, panel)) errors.push("hole outside panel");
    for (const other of holes)
      if (h !== other && overlaps(h, other)) errors.push("holes overlap");
  }
  return errors; // shown as markers on the canvas
}
```

Say aloud: "It flags holes outside the panel or overlapping each other - caught on save, before the factory floor."

JWT auth middleware with blacklist check

```
function auth(req, res, next) {
  const token = req.cookies.session;
  if (!token || isBlacklisted(token)) return res.status(401).end();
  try {
    req.user = jwt.verify(token, SECRET); // valid + not revoked
    next();
  } catch {
    res.status(401).end();
  }
}
```

Say aloud: "Every protected route verifies the JWT and rejects blacklisted (logged-out) tokens."

asyncHandler - one place for async errors

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

Say aloud: “This wraps async routes so a thrown error goes straight to the central error handler.”

Undo/redo history

```
const history = { past: [], present: state, future: [] };

function undo() {
  if (!history.past.length) return;
  history.future.unshift(history.present);
  history.present = history.past.pop();    // step back
}
```

Say aloud: “Undo pops the last state off the past and pushes the current onto the future - redo is the mirror.”

Parametrised query (node-pg)

```
const { rows } = await pool.query(
  "SELECT * FROM projects WHERE customer_id = $1", [customerId]);
```

Say aloud: “Parametrised, so there’s no SQL injection even with user input.”

The hooks to drop on purpose

1. The interactive diagram editor with real geometry validation
2. Boolean geometry (polygon difference) for cutting holes
3. Bezier curve-fitting
4. Prisma -> direct Postgres migration (and why)
5. JWT + token blacklist + TOTP 2FA
6. S3 failover file uploads

And the one to be careful with: the **AI-assisted** / **WhatsApp** diagram claim - only say it if it's genuinely built; otherwise lead with the editor and validation, which are truly yours.